



Construção de uma aplicação com arquitetura em camadas usando Express



1. Identificação da Atividade

Item	Descrição
Disciplina	Desenvolvimento de Software (Back-End/Front-End)
Unidade	2 — Frameworks
Tipo	Atividade Prática Orientada (APO)
Carga horária	10 horas
Modalidade	Individual ou dupla
Entrega	Repositório público no GitHub + README documentado



2. Contextualização

Você foi contratado(a) como desenvolvedor(a) back-end para construir a **API de um sistema de biblioteca**. A API deve permitir o gerenciamento de **livros** e **autores**, seguindo uma arquitetura em camadas com Express.

Por que esta atividade existe?

Para que você aplique, na prática, os conceitos de framework estudados na Unidade 2 — Router, Controller, Service, Repository e Middleware — em um cenário profissional real.



3. Objetivos de Aprendizagem

Ao concluir esta atividade, o estudante deverá ser capaz de:

- **Implementar** uma API RESTful com Express seguindo arquitetura em camadas;
- **Separar** responsabilidades entre Router, Controller, Service e Repository;
- **Criar** middlewares para autenticação, validação e tratamento de erros;
- **Projetar** endpoints REST para múltiplos recursos relacionados;
- **Documentar** a API com README e comentários claros.

Taxonomia de Bloom: Aplicar, Analisar, Projetar.



4. Requisitos Funcionais

A API deve gerenciar dois recursos:

Autores

Método	Rota	Ação
GET	/api/autores	Listar todos os autores
GET	/api/autores/:id	Buscar autor por ID
POST	/api/autores	Cadastrar autor
PUT	/api/autores/:id	Atualizar autor
DELETE	/api/autores/:id	Excluir autor

Livros



Método	Rota	Ação
GET	/api/livros	Listar todos os livros



5. Requisitos Não-Funcionais

- **Arquitetura em camadas:** Router → Controller → Service → Repository;
- **Middleware de log:** registrar método, URL e data/hora de cada requisição;
- **Middleware de validação:** validar dados antes de criar/atualizar;
- **Middleware de erro:** tratar erros centralmente;
- **Persistência em memória** (arrays);
- **Organização de pastas** seguindo o padrão do framework;
- **README** com documentação do projeto.



6. Estrutura Esperada

```
biblioteca-api/  
├── src/  
│   ├── routes/  
│   │   ├── autores.js  
│   │   └── livros.js  
│   ├── controllers/  
│   │   ├── autorController.js  
│   │   └── livroController.js  
│   ├── services/  
│   │   ├── autorService.js  
│   │   └── livroService.js  
│   ├── repositories/  
│   │   ├── autorRepository.js  
│   │   └── livroRepository.js  
│   └── middlewares/  
│       ├── logger.js  
│       ├── validarAutor.js  
│       ├── validarLivro.js  
│       └── errorHandler.js  
├── public/  
│   └── index.html  
├── server.js  
├── package.json  
└── README.md
```





7. Critérios de Avaliação

Critério	Peso	Descrição
Arquitetura em camadas	30%	Separação correta entre Router, Controller, Service, Repository
Funcionalidade	25%	Todas as rotas funcionando corretamente
Middlewares	20%	Logger, validação e tratamento de erros implementados
Organização do código	15%	Pastas, nomes, convenções
Documentação	10%	README completo e claro



8. Resultado Esperado

Ao finalizar a atividade, o estudante deverá ter:

- Uma API RESTful completa com 10 endpoints;
- Arquitetura em camadas com separação de responsabilidades;
- Middlewares funcionais (log, validação, erro);
- Código organizado em pastas separadas;
- README documentando o projeto.

